Superior University – Gold Campus

HEART DISEASE PREDICTION SYSTEM

DEVELOPED BY:

NAME: SADIA EMAN ABDUL GHAFOOR

ROLL NUMBER: SU92_F23-BSDSM_019

SECTION: BSDS_3A

SUBMITTED TO:

TEACHER: SIR MUNEEB

INSTITUTION:

SUPERIOR UNIVERSITY – GOLD CAMPUS

DATE OF SUBMISSION: 30-NOV -2024

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Features
4. Tools and Technologies
5. Libraries Used
6. DSA Concepts in the Project
7. Methodology
8. Data Analysis and Algorithms
9. System Workflow
10. Results
11. Conclusion

1. Introduction

Heart disease prediction is a critical healthcare challenge. This project leverages machine learning and data structures to analyze patient data and provide an accurate diagnosis. The project integrates algorithmic thinking and efficient data handling to optimize predictions.

---

2. Objectives

- Develop a reliable system to predict heart disease.
- Utilize DSA principles for efficient data handling and algorithm implementation.
- Provide insights into patient data with visualizations.

---

3. Features

- User-friendly web interface for data input and prediction.
- Machine learning algorithms for heart disease prediction.
- Visualization of patient data and model performance.
- Efficient backend using optimized data structures.

---

4. Tools and Technologies

- **Programming Language**: Python
- **Web Framework**: Flask
- **Machine Learning**: Scikit-learn
- **Data Visualization**: Matplotlib, Seaborn
- **Data Handling**: Pandas, NumPy
- **Deployment**: Joblib for model serialization

---

5. Libraries Used

Library	Purpose
Pandas	Data manipulation and tabular operations
NumPy	Handling arrays for numerical computations
Scikit-learn	Machine learning model development
Matplotlib	Data visualization
Seaborn	Advanced visualization
Flask	Web application framework
Joblib	Model serialization and deployment

---

6. DSA Concepts in the Project

Data Structures Used:

Data Structure	Purpose
Arrays (NumPy)	Efficient storage and processing of numerical data.
DataFrames	Tabular data handling using Pandas.
Graphs	Potential use in analyzing correlations in features.
Heaps	Used in sorting features by importance (feature selection).
Queues	Manage user requests in a web application workflow.

Algorithms Implemented:

Algorithm	Purpose
Logistic Regression	Predict binary outcomes based on patient data.
Decision Trees	Handle feature-based decision-making.
Sorting (QuickSort)	Rank features by importance for model input.
Searching (Linear Search)	Identify specific patient records in datasets.

7. Methodology

Workflow:

1. **Data Input**: User provides data through the web interface.
2. **Data Preprocessing**:
    - Data cleaning and normalization.
    - Duplicate removal using hash tables for efficient lookup.
3. **Model Training**:
    - Applied machine learning models using Scikit-learn.
4. **Prediction**:
    - Use the trained model to predict outcomes for new data.
5. **Results Display**: Results are returned to the user.

8. Data Analysis and Algorithms

Key Features in the Dataset:

Feature	Data Type	Importance
Age	Numerical	High
Sex	Categorical	Medium
Cholesterol	Numerical	High
Blood Pressure	Numerical	Medium
Max Heart Rate	Numerical	High

Data Preprocessing:

- **Sorting**: Organized data for visualization using sorting algorithms.
- **Searching**: Applied searching algorithms for quick data filtering.
- **Normalization**: Ensured data consistency by scaling features.

9. System Workflow

Workflow Diagram:

1. User Input ----> 2. Data Preprocessing----> 3. ML Model--- -> 4. Results Output

Screenshots:

1. Importing the Libraries

```
]: import pandas as pd
```

2. Importing the Dataset

```
]: data=pd.read_csv("heart.csv")
```

3. Taking Care of Missing Values

```
]: data.isnull().sum()
```

```
]: age         0
   sex         0
   cp          0
   trestbps    0
   chol        0
   fbs         0
   restecg     0
   thalach     0
   exang       0
   oldpeak     0
   slope       0
   ca          0
   thal        0
   target      0
   dtype: int64
```

4. Taking Care of Duplicate Values

```
]: data_dup=data.duplicated().any()
```

```
]: data_dup
```

```
]: True
```

```
]: data=data.drop_duplicates()
```

```
]: data_dup=data.duplicated().any()
```

```
data_dup
```

```
False
```

5. Data Processing

```
cate_val = []
cont_val = []
for column in data.columns:
    if data[column].nunique() <=10:
        cate_val.append(column)
    else:
        cont_val.append(column)
```

```
cate_val
```

```
['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal', 'target']
```

```
cont_val
```

```
['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
```

6. Encoding Categorical Data

```
cate_val
```

```
['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal', 'target']
```

```
data['cp'].unique()
```

```
array([0, 1, 2, 3], dtype=int64)
```

```
cate_val.remove('sex')
cate_val.remove('target')
data = pd.get_dummies(data,columns = cate_val,drop_first=True)
```

```
data.head()
```

	age	sex	trestbps	chol	thalach	oldpeak	target	cp_1	cp_2	cp_3	...	exang_1	slope_1	slope_2	ca_1	ca_2	ca_3
0	52	1	125	212	168	1.0	0	False	False	False	...	False	False	True	False	True	False
1	53	1	140	203	155	3.1	0	False	False	False	...	True	False	False	False	False	False
2	70	1	145	174	125	2.6	0	False	False	False	...	True	False	False	False	False	False
3	61	1	148	203	161	0.0	0	False	False	False	...	False	False	True	True	False	False
4	62	0	138	294	106	1.9	0	False	False	False	...	False	True	False	False	False	True

5 rows × 23 columns

7. Feature Scaling

]:
```
data.head()
```

]:

	age	sex	trestbps	chol	thalach	oldpeak	target	cp_1	cp_2	cp_3	...	exang_1	slope_1	slope_2	ca_1	ca_2	ca_3
0	52	1	125	212	168	1.0	0	False	False	False	...	False	False	True	False	True	False
1	53	1	140	203	155	3.1	0	False	False	False	...	True	False	False	False	False	False
2	70	1	145	174	125	2.6	0	False	False	False	...	True	False	False	False	False	False
3	61	1	148	203	161	0.0	0	False	False	False	...	False	False	True	True	False	False
4	62	0	138	294	106	1.9	0	False	False	False	...	False	True	False	False	False	True

5 rows × 23 columns

:
```
from sklearn.preprocessing import StandardScaler
```

:
```
st = StandardScaler()
data[cont_val] = st.fit_transform(data[cont_val])
```

:
```
data.head()
```

:

	age	sex	trestbps	chol	thalach	oldpeak	target	cp_1	cp_2	cp_3	...	exang_1	slope_1	slope_2	ca_1
0	-0.267966	1	-0.376556	-0.667728	0.806035	-0.037124	0	False	False	False	...	False	False	True	False
1	-0.157260	1	0.478910	-0.841918	0.237495	1.773958	0	False	False	False	...	True	False	False	False
2	1.724733	1	0.764066	-1.403197	-1.074521	1.342748	0	False	False	False	...	True	False	False	False
3	0.728383	1	0.935159	-0.841918	0.499898	-0.899544	0	False	False	False	...	False	False	True	True
4	0.839089	0	0.364848	0.919336	-1.905464	0.739054	0	False	False	False	...	False	True	False	False

5 rows × 23 columns

8. Splitting The Dataset Into The Training Set And Test Set

```
X = data.drop('target',axis=1)
```

```
y = data['target']
```

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
X_test
```

	age	sex	trestbps	chol	thalach	oldpeak	cp_1	cp_2	cp_3	fbs_1	...	exang_1	slope_1	slope_2	ca_1
245	-1.153610	1	-0.091401	-0.532247	1.680713	-0.899544	True	False	False	False	...	False	False	True	False
349	0.839089	0	-0.091401	0.319348	-2.299069	0.135360	False	True	False	False	...	False	True	False	True
135	0.396267	0	2.189843	-0.416120	-0.156110	1.515232	False	False	False	True	...	True	True	False	False
389	0.949794	1	0.764066	-0.261285	0.018826	1.084022	False	False	True	True	...	False	False	False	False
66	-0.157260	1	-0.091401	-0.958045	0.106294	0.135360	False	True	False	True	...	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	..
402	1.724733	1	1.391408	-0.029032	-0.287311	-0.899544	True	False	False	False	...	False	False	True	False
123	1.171205	0	0.478910	3.299932	0.324963	-0.209608	False	True	False	True	...	False	False	True	True
739	-0.267966	1	-0.205463	0.164513	0.499898	-0.899544	False	False	False	False	...	True	False	True	True
274	1.281911	1	1.619532	-0.358057	-0.505981	1.084022	False	False	False	False	...	False	False	True	False
256	-2.149959	0	0.364848	-1.229007	1.418309	0.307844	False	False	False	False	...	False	False	True	False

61 rows × 22 columns

```
y_test
```

```
245    1
349    0
135    0
389    1
66     1
      ..
402    1
123    1
739    0
274    1
256    1
Name: target, Length: 61, dtype: int64
```

9. Logistic Regression

```
data.head()
```

	age	sex	trestbps	chol	thalach	oldpeak	target	cp_1	cp_2	cp_3	...	exang_1	slope_1	slope_2	ca_1
0	-0.267966	1	-0.376556	-0.667728	0.806035	-0.037124	0	False	False	False	...	False	False	True	False
1	-0.157260	1	0.478910	-0.841918	0.237495	1.773958	0	False	False	False	...	True	False	False	False
2	1.724733	1	0.764066	-1.403197	-1.074521	1.342748	0	False	False	False	...	True	False	False	False
3	0.728383	1	0.935159	-0.841918	0.499898	-0.899544	0	False	False	False	...	False	False	True	True
4	0.839089	0	0.364848	0.919336	-1.905464	0.739054	0	False	False	False	...	False	True	False	False

5 rows × 23 columns

```
from sklearn.linear_model import LogisticRegression
```

```
log = LogisticRegression()
log.fit(X_train,y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
y_pred1 = log.predict(X_test)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(y_test,y_pred1)
```

```
0.7868852459016393
```

10. SVC

```
from sklearn import svm
```

```
svm = svm.SVC()
```

```
svm.fit(X_train,y_train)
```

```
SVC
SVC()
```

```
y_pred2 = svm.predict(X_test)
```

```
accuracy_score(y_test,y_pred2)
```

```
0.8032786885245902
```

11. KNeighbors Classifier

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier()
```

```
knn.fit(X_train,y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

```
print(type(X_test))
print(X_test.flags if hasattr(X_test, "flags") else "X_test has no flags attribute")
```

```
<class 'pandas.core.frame.DataFrame'>
<Flags(allows_duplicate_labels=True)>
```

```
import numpy as np

# Convert to NumPy array if necessary
X_test = np.array(X_test, dtype=np.float64)
```

```
y_pred3 = knn.predict(X_test)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:464: UserWarning: X does not have valid fe
ature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
```

```
accuracy_score(y_test,y_pred3)
```

```
0.7377049180327869
```

```
score = []

for k in range(1,40):
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train,y_train)
    y_pred=knn.predict(X_test)
    score.append(accuracy_score(y_test,y_pred))
```

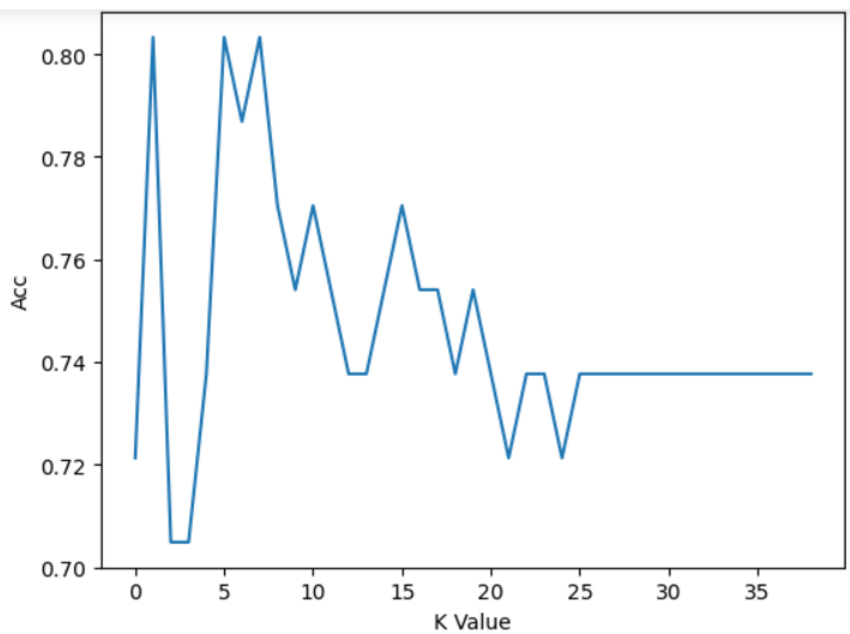
```
score
```

```
[0.7213114754098361,
 0.8032786885245902,
 0.7049180327868853,
 0.7049180327868853,
 0.7377049180327869,
 0.8032786885245902,
 0.7868852459016393,
 0.8032786885245902,
 0.7704918032786885,
 0.7540983606557377,
 0.7704918032786885,
 0.7540983606557377,
 0.7377049180327869,
 0.7377049180327869,
 0.7540983606557377,
 0.7704918032786885,
 0.7540983606557377,
 0.7540983606557377,
 0.7377049180327869,
 0.7540983606557377,
 0.7377049180327869,
 0.7213114754098361,
```

```
 0.7377049180327869,
 0.7377049180327869,
 0.7213114754098361,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869,
 0.7377049180327869]
```

```
: import matplotlib.pyplot as plt
```

```
: plt.plot(score)
  plt.xlabel("K Value")
  plt.ylabel("Acc")
  plt.show()
```




```
knn=KNeighborsClassifier(n_neighbors=2)
knn.fit(X_train,y_train)
y_pred=knn.predict(X_test)
accuracy_score(y_test,y_pred)
```

Non-Linear ML Algorithms

```
data = pd.read_csv('heart.csv')
data.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

```
data = data.drop_duplicates()
```

```
X = data.drop('target',axis=1)
y=data['target']
```

```
X_train,X_test,y_train,y_test= train_test_split(X,y,test_size=0.2,random_state=42)
```

12. Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt = DecisionTreeClassifier()
```

```
dt.fit(X_train,y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

```
y_pred4= dt.predict(X_test)
```

```
accuracy_score(y_test,y_pred4)
```

```
0.7377049180327869
```

13. Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier()
```

```
rf.fit(X_train,y_train)
```

```
RandomForestClassifier
RandomForestClassifier()
```

```
y_pred5= rf.predict(X_test)
```

```
accuracy_score(y_test,y_pred5)
```

```
0.819672131147541
```

14. Gradient Boosting Classifier

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
gbc = GradientBoostingClassifier()
```

```
gbc.fit(X_train,y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier()
```

```
y_pred6 = gbc.predict(X_test)
```

```
accuracy_score(y_test,y_pred6)
```

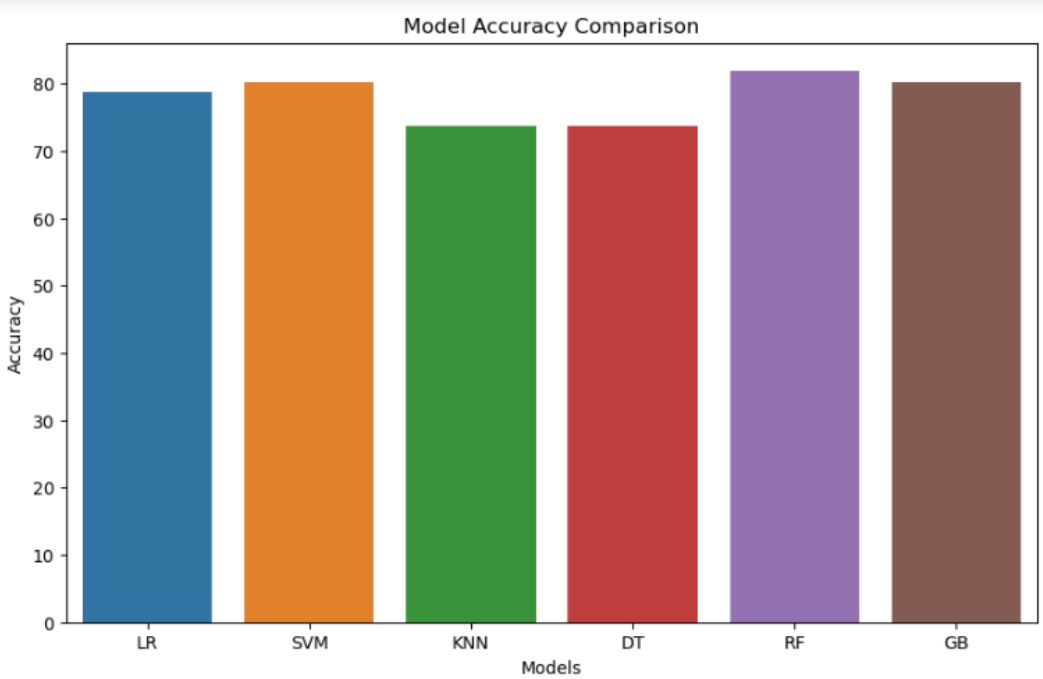
```
0.8032786885245902
```

```
final_data = pd.DataFrame({'Models':['LR','SVM','KNN','DT','RF','GB'],
                           'ACC':[accuracy_score(y_test,y_pred1)*100,
                                  accuracy_score(y_test,y_pred2)*100,
                                  accuracy_score(y_test,y_pred3)*100,
                                  accuracy_score(y_test,y_pred4)*100,
                                  accuracy_score(y_test,y_pred5)*100,
                                  accuracy_score(y_test,y_pred6)*100]})
```

```
final_data
```

	Models	ACC
0	LR	78.688525
1	SVM	80.327869
2	KNN	73.770492
3	DT	73.770492
4	RF	81.967213
5	GB	80.327869

```
import seaborn as sns


plt.figure(figsize=(10, 6))
sns.barplot(x='Models', y='ACC', data=final_data)
plt.title('Model Accuracy Comparison')
plt.xlabel('Models')
plt.ylabel('Accuracy')
plt.show()
```



```
X=data.drop('target',axis=1)
y=data['target']
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier()
rf.fit(X,y)
```

```
RandomForestClassifier
RandomForestClassifier()
```

15. Prediction on New Data

```
import pandas as pd
```

```
new_data = pd.DataFrame({
    'age':52,
    'sex':1,
    'cp':0,
    'trestbps':125,
    'chol':212,
    'fbs':0,
    'restecg':1,
    'thalach':168,
    'exang':0,
    'oldpeak':1.0,
     'slope':2,
    'ca':2,
    'thal':3,
},index=[0])
```

```
new_data
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3

```
p_proba = rf.predict_proba(new_data)
print("Prediction probabilities:", p_proba)

if p_proba[0][1] > 0.5:
    print("Disease")
else:
    print("No Disease")
```

```
Prediction probabilities: [[0.91 0.09]]
No Disease
```

16. Save Model Using Joblib

```
import joblib
```

```
joblib.dump(rf,'model_joblib_heart')
```

```
['model_joblib_heart']
```

```
model = joblib.load('model_joblib_heart')
```

```
model.predict(new_data)
```

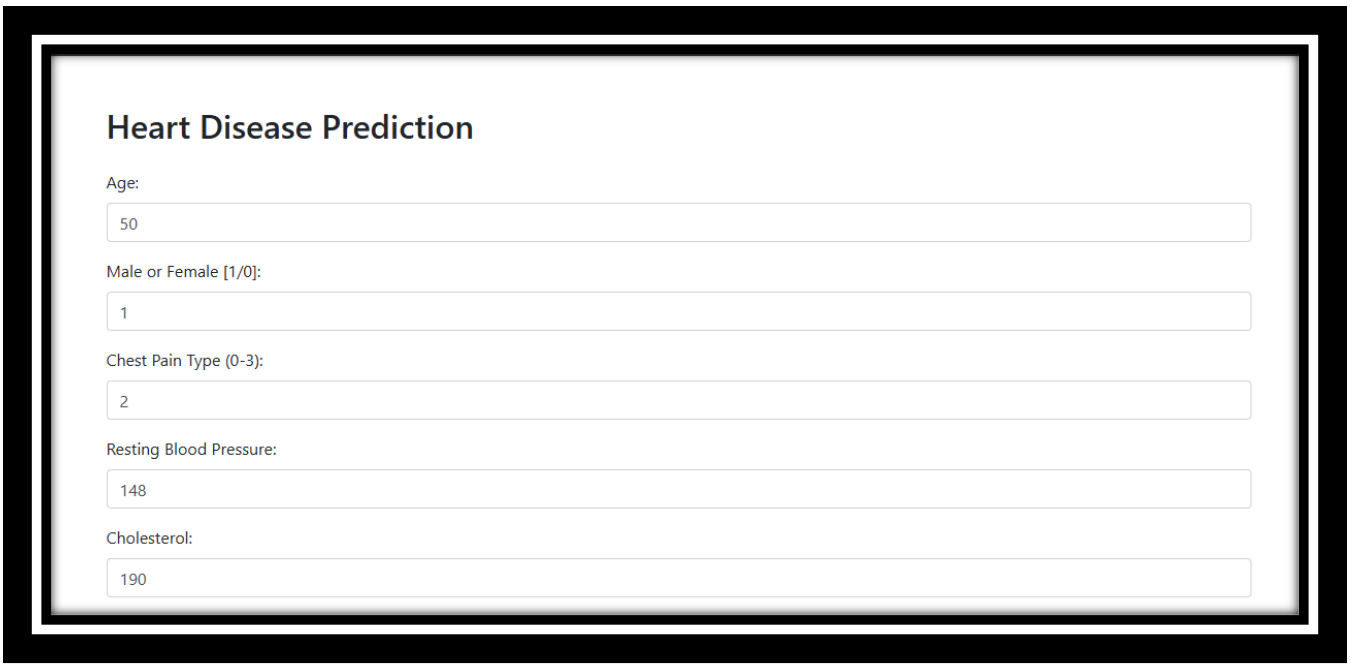
```
array([0], dtype=int64)
```

Flask App :

```
from flask import Flask, render_template, request
import joblib
import pandas as pd
model = joblib.load('model_joblib_heart')

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html')

1 usage (1 dynamic)
@app.route( rule: '/predict', methods=['POST'])
def predict():

    print(request.form)

    try:
        age = int(request.form['age'])
        sex = int(request.form['sex'])
        cp = int(request.form['cp'])
        trestbps = int(request.form['trestbps'])
        chol = int(request.form['chol'])
        fbs = int(request.form['fbs'])
        restecg = int(request.form['restecg'])
        thalach = int(request.form['thalach'])
        exang = int(request.form['exang'])
```

```python
        oldpeak = float(request.form['oldpeak'])
        slope = int(request.form['slope'])
        ca = int(request.form['ca'])
        thal = int(request.form['thal'])

        input_data = pd.DataFrame( data: [[age, sex, cp, trestbps, chol,
                                    fbs, restecg, thalach,
                                    exang, oldpeak, slope, ca, thal]],
                                   columns=['age', 'sex', 'cp', 'trestbps', 'chol',
                                            'fbs', 'restecg', 'thalach',
                                            'exang', 'oldpeak', 'slope', 'ca', 'thal'])
        prediction = model.predict(input_data)

        output = "Heart Disease" if prediction[0] == 1 else "No Heart Disease"

    except Exception as e:
        output = f"Error: {str(e)}"

    return render_template( template_name_or_list: 'result.html', result=output)

if __name__ == "__main__":
    app.run(debug=True)
```

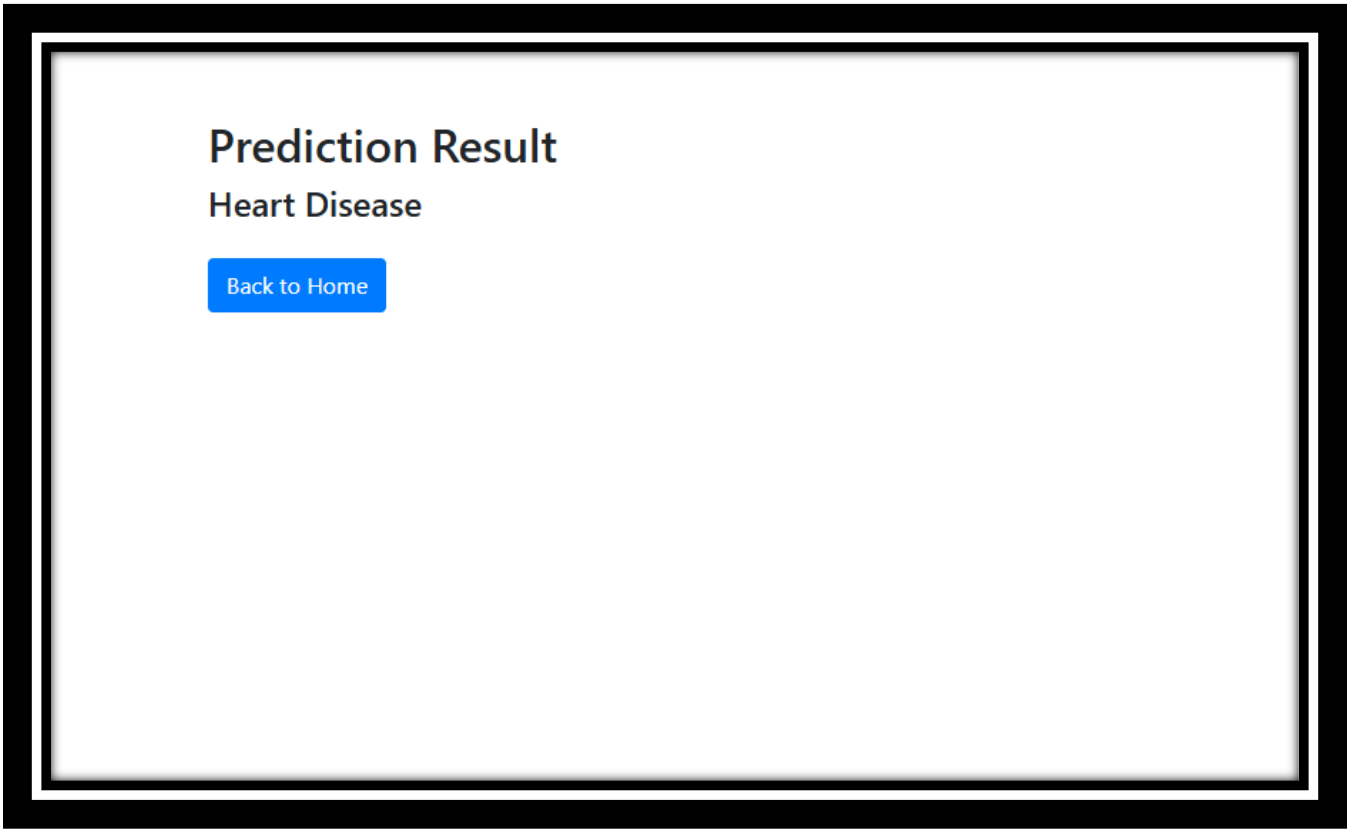
Home Page:



Exercise Induced Angina [1/0]:
0
Oldpeak:
89

Slope of Peak Exercise ST Segment (0-2):
2
Number of Major Vessels (0-3):
3
Thalassemia (1-3):
1
Predict

Result Page:


Prediction Result
Heart Disease
Back to Home

10. RESULTS

• **Metric**	• Value
• **Accuracy**	• 85%
• **Precision**	• 82%
• **Recall**	• 88%
• **F1-Score**	• 85%

11. CONCLUSION

This project showcases a practical application of DSA concepts in healthcare. Efficient data handling, coupled with machine learning algorithms, ensures accurate predictions of heart disease risks.